

# SKILLGRAPH V2

## Industry-Grade Production Plan

University Placement OS · AI Career Intelligence · Full Autonomy Stack

|                                                                                 |                                                                                  |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>OpenClaw Agent</b><br>Autonomous job application with custom resume          | <b>Faculty Placement OS</b><br>Job posting, applicant tracking, cohort analytics |
| <b>Recruiter Portal</b><br>Talent discovery from registered university students | <b>Campus Job Board</b><br>Live postings + historical status tracking            |
| <b>ATS Resume Builder</b><br>PDF resume with live ATS score optimisation        | <b>Market Benchmarks</b><br>University cohort percentile + skill demand data     |

|            |                                                             |
|------------|-------------------------------------------------------------|
| Repository | github.com/ftaakash/SkillGraph                              |
| Framework  | Next.js 16+ App Router · React 18 · TypeScript              |
| AI Engine  | Groq SDK → llama-3.3-70b-versatile (swap-ready for GPT-4o)  |
| Database   | Prisma ORM · PostgreSQL (multi-role schema)                 |
| Automation | Playwright headless · BullMQ Redis queues                   |
| Goal       | Replace university placement portals — 3-role production OS |

1. Current State Analysis — What Exists vs What's Missing

Based on the project overview and repository analysis. SkillGraph V2 has a strong architectural foundation and a working UI, but several core production features are incomplete, stubbed, or not yet connected to live data sources. This section maps every user-facing requirement to its current implementation status.

| Requirement / Feature                                   | Status      | Implementation Plan                                                                                                   |
|---------------------------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------|
| OpenClaw — Playwright scraper (Greenhouse/Lever/Soka)   | Partial     | Scraping logic exists in lib/openclaw/scrapper.ts but selectors break on dynamic portals. Needs stealth plugin.       |
| OpenClaw — Apply with customised resume                 | Partial     | Match scoring (10-dim) exists. Resume tailoring via LLM exists. PDF generation via @react-pdf/renderer.               |
| OpenClaw — Downloadable tailored resume per application | Partial     | react-pdf present. PDF generation logic written. Download endpoint /api/resume/export not fully wired.                |
| Faculty — Post job openings                             | Not Built   | FacultyJobPosting model exists in schema. POST /api/faculty/jobs handler not built. No UI page at /faculty/jobs.      |
| Faculty — Track applicants per posted job               | Not Built   | JobApplication model exists. Application pipeline UI (Applied→Shortlisted→Offered) not built. No /faculty/applicants. |
| Recruiter Panel — Talent Discovery filter by collegeId  | Not Built   | Recruiter/talent exists as stub. Query is not scoped to collegeId — returns all students globally.                    |
| Student — Market Benchmarks with university cohort data | Partial     | Benchmark page exists with bell curve. But data is global, not scoped to the student's own university.                |
| Student — Campus Job Board (posted jobs + status)       | Partial     | /jobs page stub exists. Does not query FacultyJobPosting. No historical status column showing past applications.      |
| Student — Resume Builder → ATS-scored PDF download      | Partial     | Builder UI exists. ATS scoring prompt written. PDF export not hooked to download button. No section headers.          |
| Student — Skill Gap → real-time skill demand            | Partial     | /market page shows scraped data. Not real-time — weekly batch only. No live demand indicator distinguishing by skill. |
| 3-role auth (STUDENT / FACULTY / RECRUITER)             | Partial     | UserRole enum in schema. NextAuth session extended. Middleware routing guards written but not fully implemented.      |
| Faculty — Cohort analytics dashboard                    | Not Built   | No /faculty/analytics page. No aggregation query for cohort readiness, top gaps, placement stats.                     |
| Pipeline integrity (dedup + status normalisation)       | Partial     | Anti-duplication cron designed. Not scheduled via QStash. Dedup logic not production-tested.                          |
| Production deployment (Docker / Vercel)                 | Not Started | No Dockerfile. No docker-compose. No CI/CD pipeline. No environment variable management for secrets.                  |

2. Production Architecture

The architecture preserves the existing monolithic Next.js structure while adding the missing production layers. No microservices split — single deployable unit with clearly separated concerns.

| Existing (keep unchanged)                             | New additions (production)                                   |
|-------------------------------------------------------|--------------------------------------------------------------|
| • Next.js 16+ App Router — preserve strictly          | • Playwright + stealth plugin for OpenClaw scraping          |
| • Groq llama-3.3-70b-versatile → hot-swap to GPT-4o   | • Cloudinary for resume PDF storage + versioning             |
| • Tailwind CSS v4 via @theme in globals.css           | • Upstash QStash for scheduled cron jobs (replace node-cron) |
| • Shadcn/Radix components — no additional UI libs     | • SendGrid for transactional email notifications             |
| • Prisma v5 + PostgreSQL (Neon serverless for prod)   | • Vercel for deployment (or Docker + Railway)                |
| • BullMQ + Redis (Upstash for prod) — existing queues | • GitHub Actions CI/CD pipeline                              |
| • @react-pdf/renderer — existing PDF generation       | • Naukri + Internshala APIs for real-time skill demand       |
| • NextAuth v5 JWT sessions — extend role + collegeId  | • CollegeInstitution model for university scoping            |

University Scoping Architecture

Every data query is scoped to a CollegeInstitution entity. When a faculty account is created, it is linked to a college via collegeId. All student registrations within that college are linked to the same collegeId. Recruiter talent discovery, student benchmarks, and campus job postings all filter by this key — creating isolated university tenants within the shared platform.

```
// Core scoping pattern – used across ALL university-facing queries
// Every query that should be university-scoped uses this filter:

const COLLEGE_SCOPE = { collegeId: session.user.collegeId };

// Example: Recruiter talent query scoped to one university
const students = await prisma.user.findMany({
  where: { role: 'STUDENT', isProfileVisible: true, ...COLLEGE_SCOPE },
  select: { id:true, readinessScore:true, targetRole:true, SkillProfile:true }
});

// Example: Student benchmark scoped to their university peers
const cohort = await prisma.benchmark.findMany({
  where: { collegeId: session.user.collegeId, role: student.targetRole }
});

// Example: Campus job board – only this university's postings
const jobs = await prisma.facultyJobPosting.findMany({
  where: { collegeId: session.user.collegeId, deadline: { gte: new Date() } }
});
```

3. OpenClaw — Production-Grade Autonomous Application Agent

REQUIREMENT: The OpenClaw bot must consistently apply to live jobs on portals with a customised resume. The resume must be downloadable by the student.

3.1 Playwright Scraper — Fixes Required

| Scraper fixes                                                                                                                                                                                                             | Application form logic                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Install playwright-extra + puppeteer-extra-plugin-stealth to bypass bot detection on Naukri/LinkedIn</li></ul>                                                                    | <ul style="list-style-type: none"><li>• Store scraped HTML snapshot in OpenClawListing.rawHtml for debugging failed applications</li></ul>      |
| <ul style="list-style-type: none"><li>• Add per-portal selectors: Naukri (.jobTuple), LinkedIn (.job-search-card), Greenhouse (.posting), Lever (.posting), Internshala (.job-card), Unstop (.opportunity-card)</li></ul> | <ul style="list-style-type: none"><li>• Detect CAPTCHAs: if CAPTCHA detected, mark listing status=Blocked and notify student</li></ul>          |
| <ul style="list-style-type: none"><li>• Implement retry logic: 3 attempts with 2s exponential backoff before marking listing as Failed</li></ul>                                                                          | <ul style="list-style-type: none"><li>• Resume upload: detect file input field, upload tailored PDF blob directly via Playwright</li></ul>      |
| <ul style="list-style-type: none"><li>• Rate limiting: 800ms minimum delay between requests per portal — avoid IP ban</li></ul>                                                                                           | <ul style="list-style-type: none"><li>• Confirmation detection: scan post-submit page for success/error keywords to determine outcome</li></ul> |

3.2 Customised Resume per Application — Full Flow

|                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 Student submits OpenClaw config</b><br>Sets target roles, cities, min CTC, daily limit, blacklisted companies. Config stored in OpenClawConfig model. isActive toggle controls the agent.           |
| <b>2 BullMQ worker fires at 7:00 AM IST via QStash</b><br>processOpenClawAgent(userId) queued. Worker fetches student SkillProfile + resume base data + OpenClawConfig.                                  |
| <b>3 Playwright scrapes matching listings</b><br>Per-portal scrapers extract JD text, company, role, CTC, location. Stored in OpenClawListing. Dedup check against existing OpenClawApplication records. |
| <b>4 Groq LLM scores listing (10-dimension engine)</b><br>Returns EvaluationResult with grade A–F, matchScore, tailoringNotes. Only grade C+ proceed to application.                                     |

5 Groq LLM tailors resume JSON for this specific JD

Uses RESUME\_TAILOR\_PROMPT: rewrites summary, reorders skills to match JD keywords, adjusts project descriptions. All facts preserved.

6 @react-pdf/renderer generates tailored PDF

ResumeDocument component receives tailored JSON. PDF rendered server-side. Uploaded to Cloudinary. ResumeVersion record created with cloudinaryUrl.

7 Playwright fills and submits application form

Form fields mapped: name, email, phone, portfolio, CTC fields. Resume PDF uploaded via file input. Cover letter injected. Submit button clicked. Screenshot captured.

8 Result logged to OpenClawApplication

Status: Applied / Failed / Blocked. resumeVersionId linked. matchScore and tailoringNotes stored. Student sees live feed on /openclaw dashboard.

3.3 Resume Download — Implementation

Every application creates a versioned ResumeVersion record with a Cloudinary URL. The student can download any tailored resume from the OpenClaw dashboard or the Resume Builder history.

```
// GET /api/resume/download/[versionId] — signed download URL
export async function GET(req, { params }) {
  const session = await getServerSession(authOptions);
  const version = await prisma.resumeVersion.findUnique({
    where: { id: params.versionId, userId: session.user.id } // own records only
  });
  if (!version) return new Response('404', { status: 404 });

  // Option A: redirect to Cloudinary signed URL (simplest)
  return NextResponse.redirect(version.cloudinaryUrl);

  // Option B: stream PDF bytes (for custom filename)
  const pdf = await fetch(version.cloudinaryUrl);
  const blob = await pdf.arrayBuffer();
  return new Response(blob, {
    headers: {
      'Content-Type': 'application/pdf',
      'Content-Disposition': `attachment; filename="Resume_${version.targetRole}_${version.createdAt}.pdf"`
    }
  });
}
```

4. Faculty Placement Cell OS — Job Posting + Applicant Tracking

REQUIREMENT: Faculty/Recruiter must be able to post jobs and track applicants. Applicant status pipeline: Applied → Shortlisted → OA Sent → Interview → Offered → Placed.

| Job posting (faculty)                                                                                                                                                                                                   | Applicant tracking (faculty)                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• POST /api/faculty/jobs — create job posting with: company, role, CTC band (■ LPA), min CGPA, eligible branches, application deadline, drive date, process description</li></ul> | <ul style="list-style-type: none"><li>• GET /api/faculty/applications?jobId=X — all applicants for a job</li></ul>            |
| <ul style="list-style-type: none"><li>• GET /api/faculty/jobs — list all postings for this college (collegeld scoped)</li></ul>                                                                                         | <ul style="list-style-type: none"><li>• PATCH /api/faculty/applications/[id] — move student through pipeline stages</li></ul> |

|                                                                                                                                |                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• PATCH /api/faculty/jobs/[id] — edit or close a posting</li> </ul>                     | <ul style="list-style-type: none"> <li>• Pipeline stages: Applied → Shortlisted → OA Sent → Interview → Offered → Placed → Rejected</li> </ul> |
| <ul style="list-style-type: none"> <li>• Students notified via email (SendGrid) when a new eligible posting appears</li> </ul> | <ul style="list-style-type: none"> <li>• Status change triggers SendGrid email to student with next steps</li> </ul>                           |
| <ul style="list-style-type: none"> <li>• Job card visible on /faculty/jobs page with application count badge</li> </ul>        | <ul style="list-style-type: none"> <li>• On "Placed": creates PlacementRecord for NAAC/NBA reporting export</li> </ul>                         |

## 4.1 Faculty Dashboard Widgets — /faculty/dashboard

|                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A Overview stats</b><br>Total registered students (collegeld scoped), avg cohort readiness score, open job postings, applications this week, confirmed placements this year.     |
| <b>B Cohort skill heatmap</b><br>Top 10 skills across all students (SkillProfile aggregation) vs top 10 market demands (JobPosting/IndiaMarketData). Built with Recharts bar chart. |
| <b>C Placement funnel</b><br>Pipeline stage counts across all active jobs: Applied / Shortlisted / Interview / Offered / Placed. Funnel chart from Recharts.                        |
| <b>D Readiness distribution</b><br>Bell curve of all students' readinessScore by targetRole. Identifies which cohort segments need intervention.                                    |
| <b>E Company contact manager</b><br>/faculty/companies — manage recruiter company relationships. HR email, past drives, notes. CSV upload for bulk add.                             |
| <b>F Report export</b><br>/faculty/reports — generate CSV and PDF of all PlacementRecord rows for NAAC/NBA accreditation submission.                                                |

```
// POST /api/faculty/jobs — create job posting
export async function POST(req) {
  const session = await getServerSession(authOptions);
  if (session.user.role !== 'FACULTY') return new Response('403', {status:403});

  const { title, company, description, ctcMin, ctcMax, minCgpa,
    eligibleBranches, deadline, driveDate, processDescription } = await req.json();

  const job = await prisma.facultyJobPosting.create({
    data: {
      collegeId: session.user.collegeId, // always university-scoped
      postedBy: session.user.id,
      title, company, description,
      ctcMin, ctcMax, minCgpa,
      eligibleBranches, // ['CSE','IT','ECE']
      deadline: new Date(deadline),
      driveDate: driveDate ? new Date(driveDate) : null,
      processDescription,
      status: 'OPEN'
    }
  });

  // Notify eligible students in the same college
  const eligible = await prisma.user.findMany({
    where: {
      role: 'STUDENT', collegeId: session.user.collegeId,
      branch: { in: eligibleBranches },
    }
  });
}
```

```
cgpa: { gte: minCgpa ?? 0 }
},
select: { email: true, name: true }
});
await sendBulkEmail(eligible, 'new_job_posting', { job });
return Response.json(job);
}
```

5. Recruiter Portal — Talent Discovery from University Students

REQUIREMENT: Talent Discovery should be filled with registered students of the university — not all platform students globally. Privacy-first: anonymised until mutual interest.

The critical missing piece in the current implementation: the talent query is not scoped to a specific university. Every recruiter should only see students from the university/universities they are associated with, or students who have explicitly made their profile visible to external recruiters.

| Access control                                                                                                                 | Talent card design                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Recruiter account linked to Company entity (companyId)</li></ul>                         | <ul style="list-style-type: none"><li>Talent card: anonymised ID (SG-XXXXX), readiness %, top 6 skills, project title only</li></ul> |
| <ul style="list-style-type: none"><li>Company must be verified by faculty/admin before talent access granted</li></ul>         | <ul style="list-style-type: none"><li>No name, email, college name, phone until mutual reveal</li></ul>                              |
| <ul style="list-style-type: none"><li>Recruiter can be granted access to one or more universities by faculty admin</li></ul>   | <ul style="list-style-type: none"><li>Shortlist → student notified → student accepts reveal → PII unlocked</li></ul>                 |
| <ul style="list-style-type: none"><li>RecruiterCollegeAccess junction table: which colleges a recruiter can see</li></ul>      | <ul style="list-style-type: none"><li>Recruiter can post jobs visible to opted-in students across their linked colleges</li></ul>    |
| <ul style="list-style-type: none"><li>Filter panel: target role, min readiness score, skills, year, min CGPA, branch</li></ul> | <ul style="list-style-type: none"><li>Pipeline kanban: Discovered → Interested → Revealed → Interview → Offered → Hired</li></ul>    |

```
// GET /api/recruiter/talent - university-scoped, anonymised
export async function GET(req) {
  const session = await getServerSession(authOptions);
  if (session.user.role !== 'RECRUITER' || !session.user.isVerified)
    return new Response('403', {status:403});

  // Get colleges this recruiter has access to
  const access = await prisma.recruiterCollegeAccess.findMany({
    where: { recruiterId: session.user.id, status: 'APPROVED' },
    select: { collegeId: true }
  });

  const allowedColleges = access.map(a => a.collegeId);

  const { role, minScore, skills, year, minCgpa } = parseQuery(req.nextUrl.searchParams);

  const students = await prisma.user.findMany({
    where: {
      role: 'STUDENT',
      isProfileVisible: true,
      collegeId: { in: allowedColleges }, // ← university scoping
      targetRole: role ? { contains: role, mode: 'insensitive' } : undefined,
      readinessScore: { gte: Number(minScore) || 0 },
      cgpa: minCgpa ? { gte: Number(minCgpa) } : undefined,
      year: year ? Number(year) : undefined,
    }
  });

  return Response.json(students);
}
```

```
SkillProfile: skills?.length ? { some: { skillName: { in: skills } } } : undefined
},
select: {
  id: true, readinessScore: true, targetRole: true,
  year: true, branch: true, cgpa: true,
  name: false, email: false, college: false, // NEVER returned
  SkillProfile: { select: { skillName: true, proficiency: true }, take: 8 }
}
});

// Hash IDs → SG-XXXXX and return anonymised profiles
return Response.json(anonymiseProfiles(students, allowedColleges));
}
```

6. Campus Job Board — Live + Historical Postings with Status

REQUIREMENT: Campus Job Board must show currently posted jobs AND previously posted jobs with their status. Students must see their own application status per job.

| Job listing display                                                                                                                       | Application tracking                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Query: FacultyJobPosting WHERE collegeId = student.collegeId (all — open AND closed)</li></ul>    | <ul style="list-style-type: none"><li>• Per-job application status badge: Not Applied / Applied / Shortlisted / OA Sent / Interview / Offered / Placed / Rejected</li></ul> |
| <ul style="list-style-type: none"><li>• Job card shows: company logo, role, CTC band, deadline countdown OR "Closed X days ago"</li></ul> | <ul style="list-style-type: none"><li>• Apply button: only visible if job is OPEN, student is eligible, and not already applied</li></ul>                                   |
| <ul style="list-style-type: none"><li>• Eligibility check: green tick if student meets CGPA + branch criteria, red cross if not</li></ul> | <ul style="list-style-type: none"><li>• Apply flow: select resume version (from builder) → optional cover letter → submit</li></ul>                                         |
| <ul style="list-style-type: none"><li>• AI match score per job: GPT overlay of student SkillProfile vs job required skills</li></ul>      | <ul style="list-style-type: none"><li>• Application tracker widget on /dashboard: shows last 5 applications + current stage</li></ul>                                       |
| <ul style="list-style-type: none"><li>• Filter tabs: Open / Closed / Applied / All — with counts per tab</li></ul>                        | <ul style="list-style-type: none"><li>• SendGrid notification when faculty updates application status</li></ul>                                                             |

```
// GET /api/jobs – all university jobs (open + closed) with student's application status
export async function GET(req) {
  const session = await getServerSession(authOptions);
  const studentId = session.user.id;

  const jobs = await prisma.facultyJobPosting.findMany({
    where: { collegeId: session.user.collegeId }, // university scoped
    orderBy: [{ status: 'asc' }, { createdAt: 'desc' }],
    include: {
      _count: { select: { Applications: true } },
      Applications: {
        where: { userId: studentId }, // only this student's application
        select: { id: true, status: true, appliedAt: true, resumeVersionId: true }
      }
    }
  });

  // Enrich with AI match score per job (cached 24h to avoid repeated LLM calls)
  const enriched = await Promise.all(jobs.map(async job => {
    const cached = await redis.get(`match:${studentId}:${job.id}`);
    const matchScore = cached ?? await computeMatchScore(job, session.user.id);
```

```
if (!cached) await redis.setex(`match:${studentId}:${job.id}`, 86400, matchScore);
return { ...job, matchScore, myApplication: job.Applications[0] ?? null };
});
return Response.json(enriched);
}
```

7. ATS Resume Builder — Optimised PDF with Downloadable Output

REQUIREMENT: Resume Builder must give students an ATS-score optimised resume in PDF format based on their inputs. The PDF must be downloadable.

| Editor features                                                                                                                                                                                     | ATS scoring + download                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Two entry modes: (A) Generate from scratch using SkillProfile + education data (B) Enhance uploaded PDF — paste target JD, receive improved version</li></ul> | <ul style="list-style-type: none"><li>ATS score breakdown: Keyword Match 0-40pt, Format 0-20pt, Quantification 0-20pt, Action Verbs 0-10pt, Completeness 0-10pt</li></ul>                          |
| <ul style="list-style-type: none"><li>Section editors: Summary, Skills (pulled from SkillProfile), Projects, Experience, Education, Certifications</li></ul>                                        | <ul style="list-style-type: none"><li>Keywords added (green) vs missing (red) shown as chip list</li></ul>                                                                                         |
| <ul style="list-style-type: none"><li>"AI Rewrite" per section: submits section text + target JD to Groq for ATS-optimised rewrite</li></ul>                                                        | <ul style="list-style-type: none"><li>"Download PDF" button: calls @react-pdf/renderer server-side → returns PDF blob → triggers browser download with filename Resume_{Role}_{Date}.pdf</li></ul> |
| <ul style="list-style-type: none"><li>Live ATS preview panel: updates score as student edits each section</li></ul>                                                                                 | <ul style="list-style-type: none"><li>All versions saved as ResumeVersion records — selectable when applying to campus jobs</li></ul>                                                              |

7.1 PDF Generation + Download Endpoint

```
// POST /api/resume/generate — build + return PDF
export async function POST(req) {
  const session = await getServerSession(authOptions);
  const { resumeData, targetJd, versionName } = await req.json();

  // Step 1: Groq LLM scores and enriches the resume JSON
  const scored = await groqClient.chat.completions.create({
    model: 'llama-3.3-70b-versatile',
    temperature: 0,
    messages: [
      { role: 'system', content: ATS_SCORE_PROMPT },
      { role: 'user', content: JSON.stringify({ resume: resumeData, jd: targetJd }) }
    ]
  });

  const { atsScore, keywordsUsed, keywordsMissed, sections } =
    JSON.parse(scored.choices[0].message.content);

  // Step 2: Generate PDF using @react-pdf/renderer (server-side)
  const { renderToBuffer } = await import('@react-pdf/renderer');
  const pdfBuffer = await renderToBuffer(
    <ResumeDocument data={{ ...resumeData, ...sections }} />
  );

  // Step 3: Upload to Cloudinary
  const uploadResult = await cloudinary.uploader.upload_stream(
    { resource_type: 'raw', folder: 'skillgraph/resumes',
      public_id: `resume_${session.user.id}_${Date.now()}` },
    pdfBuffer
  );
}
```



```
);

// Step 4: Save version record
await prisma.resumeVersion.create({
  data: {
    userId: session.user.id,
    cloudinaryUrl: uploadResult.secure_url,
    atsScore,
    targetJd,
    generatedBy: 'AI',
    versionName
  }
});

// Step 5: Stream PDF back to browser for immediate download
return new Response(pdfBuffer, {
  headers: {
    'Content-Type': 'application/pdf',
    'Content-Disposition': `attachment; filename="Resume_${versionName}.pdf"`
  }
});
}
```

8. Market Benchmarks — University Cohort Percentile + Skill Demand

REQUIREMENT: Market Benchmarks must show statistics of students from the same university. Students must see their rank within their own university cohort. Skill Demand must show real-time data.

| University cohort benchmark                                                                                                                               | Real-time skill demand                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Bell curve scoped to collegeld + targetRole — shows only university peers</li></ul>                                 | <ul style="list-style-type: none"><li>Real-time skill demand: live Naukri/Internshala API call on page load for student's targetRole</li></ul>    |
| <ul style="list-style-type: none"><li>Student's percentile rank within their university: "You are ahead of 72% of CSE students at your college"</li></ul> | <ul style="list-style-type: none"><li>Week-over-week delta shown with ↑↓ arrows and percentage change</li></ul>                                   |
| <ul style="list-style-type: none"><li>Readiness score history: line chart showing improvement over last 8 weeks</li></ul>                                 | <ul style="list-style-type: none"><li>Company tier demand: which tier (FAANG/Unicorn/Service) is hiring for which skills most this week</li></ul> |
| <ul style="list-style-type: none"><li>Top 10% skill profile for their role within university — what peers with 85+ readiness score have</li></ul>         | <ul style="list-style-type: none"><li>City-wise demand overlay: Bangalore vs Hyderabad vs Chennai vs Remote for student's target role</li></ul>   |
| <ul style="list-style-type: none"><li>"Climb the curve" CTA: shows exact skills to add to move from current percentile to next decile</li></ul>           | <ul style="list-style-type: none"><li>Skill demand feeds into gap analysis — skills trending up weighted higher in urgency</li></ul>              |

```
// GET /api/benchmark - university-scoped cohort stats + live skill demand
export async function GET(req) {
  const session = await getSession(authOptions);
  const student = await prisma.user.findUnique({
    where: { id: session.user.id },
    select: {
      readinessScore: true,
      targetRole: true,
      collegeId: true
    }
  });

  // University cohort benchmark (PostgreSQL percentile query)
  const cohortData = await prisma.$queryRaw`
  SELECT
    readiness_score,
    PERCENT_RANK() OVER (
      PARTITION BY college_id, target_role
      ORDER BY readiness_score
    ) as percentile,
    COUNT(*) OVER (PARTITION BY college_id, target_role) as cohort_size
  FROM "User"
  `}
```

```
WHERE college_id = ${student.collegeId}
AND target_role = ${student.targetRole}
AND role = 'STUDENT'
;

// Real-time skill demand from Naukri API (cached 1 hour)
const demandCacheKey = `demand:${student.targetRole}:${new Date().toISOString().slice(0,13)}`;
let skillDemand = await redis.get(demandCacheKey);
if (!skillDemand) {
  skillDemand = await fetchNaukriDemand(student.targetRole);
  await redis.setex(demandCacheKey, 3600, JSON.stringify(skillDemand)); // 1hr cache
}

return Response.json({ cohortData, skillDemand: JSON.parse(skillDemand),
studentScore: student.readinessScore });
}
```

## 9. Complete Prisma Schema — New + Updated Models

All existing models preserved. Run `npx prisma migrate dev --name v2_production` after adding these models. All additions are backwards-compatible.

| Model / Field          | Type & Constraints                                                                                                                            | Purpose                                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| User (updated)         | collegeld String?, cgpa Float?, branch String?, isProfileVisible Boolean? @default(false)                                                     | Field visible to link @ default false specific fields. isProfileVisible = opt-in for recruiter |
| CollegeInstitution     | id, name, city, state, tier (Tier1/Tier2/Tier3), naacGrade String?, website String?, createdAt DateTime                                       | University Entry. A student + faculty accounts link here via collegeld.                        |
| Company                | id, name, website, tier, verificationStatus (Pending/Approved), createdAt DateTime                                                            | Provide Recruiters to get hired by faculty admin before talent access granted                  |
| RecruiterCollegeAccess | id, recruiterId, collegeld, status (Pending/Approved), grantedBy (facultyId), grantedAt DateTime                                              | As a recruiter can access. Faculty grants access.                                              |
| FacultyJobPosting      | id, collegeld, postedBy (userId), title, company, description, createdAt DateTime, tier Boolean?, naacGrade Float?, eligibleForStudy Boolean? | Can post Job Posting by Tier Boolean? naacGrade Float? eligibleForStudy Boolean?               |
| JobApplication         | id, userId, jobPostingId, resumeVersionId?, coverLetter String?, applicationStatus (Applied/Shortlisted/Cancelled/Offered/Placed/Rejected)    | Student Application (Applied/Shortlisted/Cancelled/Offered/Placed/Rejected)                    |
| PlacementRecord        | id, collegeld, studentId, company, role, ctclpa Float, finalDateOfJoining DateTime, salaryOffered Float, naacGradeBy (cats/CS)                | Final Date of Joining, Salary Offered, NAAC Grade By (cats/CS) bulk upload support             |
| ResumeVersion          | id, userId, cloudinaryUrl, atsScore Float?, targetJd @unique, createdAt DateTime, versionId Integer?, isDeleted Boolean?                      | @unique, createdAt DateTime, versionId Integer?, isDeleted Boolean? Linked to JobPosting       |
| EvaluationResult       | id, listingId @unique, userId, totalScore Float, grade String, createdAt DateTime                                                             | Score dimension score result per Open Class Listing. Key Score, Spring, July, Winter           |
| InterviewStory         | id, userId, theme String, applicableRoles String[], createdAt DateTime, status (Active/Completed/Cancelled)                                   | Story where feat task stories, consists of Post, Open Class Evaluation (on On Test), cross     |
| NegotiationScript      | id, applicationId @unique, script1 Json, script2 Json, createdAt DateTime                                                                     | Script 3 Negotiation scripts generated when application status moves to Offered.               |
| IndiaMarketData        | id, skill, role, city, demandCount Int, avgCtclpa Float, createdAt DateTime                                                                   | Weekly India specific, market Intelligence. Separate from global JobPosting table              |

## 10. Complete File Structure — New Files Only

```
skillgraph-web/
  app/
    (app)/ Existing student routes
      openclaw/page.tsx EXTEND: add download button, batch eval tab
      resume-builder/page.tsx EXTEND: wire download + ATS breakdown
      jobs/page.tsx EXTEND: show open+closed + application status
      benchmark/page.tsx EXTEND: scope to collegeId + add real-time demand
      market/page.tsx EXTEND: add live demand indicator
    (faculty)/ NEW route group
      layout.tsx FacultyLayout with role-specific sidebar
```

- ■ ■■ faculty/
- ■ ■■ dashboard/page.tsx Cohort stats + skill heatmap + placement funnel
- ■ ■■ jobs/
- ■ ■■ page.tsx All posted jobs with applicant counts
- ■ ■■ new/page.tsx Create new job posting form
- ■ ■■ jobs/[id]/page.tsx Job detail + applicant pipeline kanban
- ■ ■■ applications/page.tsx All applications across all jobs
- ■ ■■ analytics/page.tsx Cohort analytics + readiness distribution
- ■ ■■ reports/page.tsx NAAC/NBA placement report export
- ■ ■■ companies/page.tsx Recruiter company management
- ■■ (recruiter)/ NEW route group
- ■ ■■ layout.tsx RecruiterLayout
- ■ ■■ recruiter/
- ■ ■■ dashboard/page.tsx Hiring funnel stats
- ■ ■■ talent/page.tsx University-scoped talent discovery
- ■ ■■ shortlist/page.tsx Saved anonymised profiles
- ■ ■■ pipeline/page.tsx Kanban: Interested→Revealed→Interview→Hired
- ■ ■■ jobs/
- ■ ■■ page.tsx Recruiter-posted jobs
- ■ ■■ new/page.tsx Post job for matched students
- ■■ api/
- ■■ faculty/
- ■ ■■ jobs/route.ts GET (list) + POST (create)
- ■ ■■ jobs/[id]/route.ts PATCH (edit/close) + DELETE
- ■ ■■ applications/route.ts GET all + PATCH status
- ■ ■■ analytics/route.ts Cohort aggregations
- ■ ■■ reports/route.ts PlacementRecord CSV/PDF export
- ■■ recruiter/
- ■ ■■ talent/route.ts Scoped anonymised talent query
- ■ ■■ reveal/route.ts Mutual-interest reveal flow
- ■ ■■ shortlist/route.ts CRUD shortlist
- ■ ■■ jobs/route.ts Post + manage recruiter jobs
- ■ ■■ verify/route.ts Company verification request
- ■■ jobs/
- ■ ■■ route.ts GET campus jobs (open + closed) for student
- ■ ■■ apply/route.ts POST job application
- ■■ resume/
- ■ ■■ generate/route.ts POST build+score → stream PDF
- ■ ■■ download/[id]/route.ts GET signed Cloudinary URL
- ■ ■■ versions/route.ts GET all saved versions
- ■■ benchmark/route.ts EXTEND: collegeId scope + skill demand
- ■■ openclaw/
- ■■ pipeline/route.ts URL → full eval + PDF (already exists, extend)
- ■■ integrity/route.ts Dedup + status normalise cron
- 
- ■■ components/
- ■■ faculty/
- ■ ■■ JobPostForm.tsx Create/edit job posting form (Shadcn)
- ■ ■■ ApplicantPipeline.tsx Kanban pipeline per job
- ■ ■■ CohortHeatmap.tsx Recharts bar: student skills vs market
- ■ ■■ PlacementFunnel.tsx Recharts funnel chart

- ■ ■ ■ recruiter/
- ■ ■ ■ TalentCard.tsx Anonymised profile card
- ■ ■ ■ CollegeAccessGate.tsx Pending verification state
- ■ ■ ■ PipelineKanban.tsx @dnd-kit/core drag-drop kanban
- ■ ■ ■ resume-builder/
- ■ ■ ■ SectionEditor.tsx Per-section AI rewrite UI
- ■ ■ ■ AtsScorePanel.tsx Live ATS breakdown sidebar
- ■ ■ ■ DownloadButton.tsx PDF download trigger
- ■ ■ ■ openclaw/
- ■ ■ ■ ApplicationFeed.tsx EXTEND: add download resume per row
- 
- ■ ■ ■ lib/
- ■ ■ ■ openclaw/
- ■ ■ ■ scraper.ts EXTEND: stealth, retry, per-portal selectors
- ■ ■ ■ scorer.ts 10-dim scoring (extend existing)
- ■ ■ ■ batch-processor.ts BullMQ concurrency 5 sub-agents
- ■ ■ ■ resume/
- ■ ■ ■ builder.ts Groq resume generation
- ■ ■ ■ enhancer.ts JD-targeted enhancement
- ■ ■ ■ pdf-renderer.ts @react-pdf/renderer server wrapper
- ■ ■ ■ market/
- ■ ■ ■ india-demand.ts Naukri API + 1hr Redis cache

11. Build Phases — Prioritised Execution Order

| Phase 1 Wks 1–2<br>Foundation                                                                                                                                                                                     | Phase 2 Wks 3–4<br>Faculty OS                                                                                                                                                            | Phase 3 Wks 5–6<br>Campus Board                                                                                                                                                                        | Phase 4 Wks 7–8<br>Recruiter                                                                                                                                                        | Phase 5 Wks 9–10 AI<br>Modules                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>→ Prisma migration: all new models</div> <div>→ NextAuth role extension</div> <div>→ Middleware routing guards</div> <div>→ CollegeInstitution + register flow</div> <div>→ Faculty + Recruiter signup</div> | <div>→ POST /api/faculty/jobs</div> <div>→ Faculty job listing page</div> <div>→ /faculty/jobs/new form</div> <div>→ Applicant pipeline kanban</div> <div>→ SendGrid notifications</div> | <div>→ GET /api/jobs (scoped+status)</div> <div>→ /jobs page open+closed</div> <div>→ Apply flow with resume select</div> <div>→ Application status badges</div> <div>→ Dashboard tracker widget</div> | <div>→ College scoped talent query</div> <div>→ Anonymisation + reveal flow</div> <div>→ RecruiterCollegeAccess</div> <div>→ Talent discovery UI</div> <div>→ Pipeline kanban</div> | <div>→ Resume builder + download</div> <div>→ ATS scoring panel</div> <div>→ OpenClaw scraper fixes</div> <div>→ Benchmark cohort scoping</div> <div>→ Real-time skill demand</div> |

Immediate first steps — do in this exact order:

- 1

Add CollegeInstitution model to schema.prisma + collegeld on User

This is the foundation for everything. Run: npx prisma migrate dev --name add\_college\_institution. Test that existing student login still works.
- 2

Update NextAuth session with role + collegeld

Modify auth.ts jwt callback. Extend types/next-auth.d.ts. Test: log session in /api/auth/session and verify role + collegeld are present.
- 3

Test middleware.ts guards for /faculty/\* and /recruiter/\*

Create stub pages at /faculty/dashboard and /recruiter/dashboard. Verify student cannot access them. Verify faculty cannot access /recruiter/\*.
- 4

Build POST /api/faculty/jobs and the /faculty/jobs/new form

Simplest real feature. Faculty can post a job. Test by creating one posting and verifying it appears in FacultyJobPosting table with correct collegeld.

5 Update GET /api/jobs to query FacultyJobPosting (not the old scraper-based JobPosting)

Students should now see their campus jobs. Test with a faculty account posting a job and a student account in the same college seeing it on /jobs.

6 Wire resume PDF download button

Simplest AI feature to fix. Connect the existing PDF generation to a download endpoint. Students get immediate, tangible value from this.

12. Production Deployment Plan

| Hosting stack (Vercel + serverless)                                                                                                                  | Secrets + CI/CD + browser automation                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Vercel deployment: connect GitHub repo → auto-deploy on push to main → zero config for Next.js</li></ul>     | <ul style="list-style-type: none"><li>• Environment variables: NEXTAUTH_SECRET, DATABASE_URL, GROQ_API_KEY, CLOUDINARY_URL, UPSTASH_REDIS_REST_URL, SENDGRID_API_KEY, QSTASH_TOKEN</li></ul> |
| <ul style="list-style-type: none"><li>• Neon PostgreSQL: serverless Postgres, scales to 0 when idle — free tier for dev, Neon Pro for prod</li></ul> | <ul style="list-style-type: none"><li>• GitHub Actions CI: lint → type-check → prisma validate → deploy on push to main</li></ul>                                                            |
| <ul style="list-style-type: none"><li>• Upstash Redis: serverless Redis for BullMQ + session cache + skill demand cache</li></ul>                    | <ul style="list-style-type: none"><li>• Playwright in production: use browserless.io or Apify as managed headless browser service — avoids self-hosting Chromium on Vercel</li></ul>         |
| <ul style="list-style-type: none"><li>• Upstash QStash: replace node-cron for OpenClaw daily trigger + pipeline integrity cron</li></ul>             | <ul style="list-style-type: none"><li>• Rate limiting: Upstash Ratelimit on all AI endpoints (10 req/min per user)</li></ul>                                                                 |
| <ul style="list-style-type: none"><li>• Cloudinary: free tier sufficient for 10,000 resume PDFs — upgrade on scale</li></ul>                         |                                                                                                                                                                                              |

```
# .github/workflows/deploy.yml — CI/CD pipeline
name: Deploy SkillGraph V2
on:
  push:
    branches: [main]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
      - run: npx prisma validate
      - run: npx prisma generate
  deploy:
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: amondnet/vercel-action@v25
    with:
      vercel-token: ${ secrets.VERCEL_TOKEN }
      vercel-org-id: ${ secrets.VERCEL_ORG_ID }
      vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
```

```
vercel-args: '--prod'
```

Playwright in production: Do NOT use Playwright directly on Vercel serverless functions — serverless functions cannot run Chromium. Use browserless.io (\$9/mo for 200 browser hours) or self-host on a Railway/Render background service. The OpenClaw BullMQ worker runs on Railway, not Vercel.